

threads share a common address space. They can also be used by processes, but then the memory in which they are stored needs to be shared between processes, like say, *Shared Memory*. We will not go into the details of POSIX memory-based implementations at this stage.

Example: Sys V IPC object: message queue

Now, let's implement the sender-receiver code using a System V message queue. Here's the algorithm.

Sender

```
Generate key (say K) using ftok library function
Create/Get Access to the Message Queue (The key K is needed here)
While (1) {
    Read from keyboard.
    Create a message.
    Send the message to the message queue
    If user has entered "exit" break the loop.
}
Delete message queue and exit program.
```

Receiver

```
Generate key (say K1)
Create/Get Access to the Message Queue (The key K1 is needed here)
While (1) {
    Read data from the Message Queue.
    Print data on the screen
    If received message is "exit" then break loop.
}
Program terminates.
```

The C programs (*msg_sender.c* and *msg_receiver.c*) can be downloaded from the e-download archive. Compile them as before and run in two terminal windows. You can use the *ipcs* command to list System V IPC objects on the system and check that the message queue has been created.

Now, let's understand the code. We used the *ftok* function to generate a key (a number), which should be unique and available to all programs that want to refer to a particular message queue. The choice of arguments (the filename '*msg_receiver.c*' and '*B*') is completely arbitrary, but you should use the same parameter in all programs using the queue. Also, the first argument should be an existing file. (You might use a file name like '*test*' before running the code, and then create the file by running the command *touch test* in the directory where you're executing the program.)

Creating or accessing an existing message queue is done by the *msgget* function, which takes the key value

and some additional information. In this example, use the permission bits with the predefined constant *IPC_CREAT*, joining them with the *or* operator. The *IPC_CREAT* constant creates the queue if it does not exist. The return value is the *message queue descriptor*, which then can be used as an argument to functions operating on the queue (reading from it, writing to it or deleting it).

Note the formation of a message. It is a C language structure:

```
struct msbuf {
    long mtype;
    char mstore [80];
}
```

The first element must be of the *long* type, and it can have any type of subsequent elements. In my example, the second element is an 80-character array.

An important feature of Sys V message queues is that the receiver can selectively retrieve messages based on the first element of the message structure. This feature is not available in pipes or FIFO, where readers can only read data in first-in-first-out order.

The functions *msgsnd*, *msgrcv* and *msgctl* send and receive messages, and delete the queue respectively. I have explained the *msgsnd* arguments in code comments. They are similar to those of the *msgrcv* function, which waits for the arrival of a message in the queue; whenever it receives a message, it returns from its blocked state. Note that the original message is removed from the queue when the *msgrcv* function returns.

If multiple messages are available in the queue, which will be retrieved? Based on the fourth argument to *msgrcv*, the type fields of messages in the queue are compared, to decide which message to retrieve. In our example, the value is 1, so the first message with a type value of 1 will be retrieved. There can be other retrieval schemes based on the fourth argument, for which you can consult the man page.

Finally, we have used the *msgctl* function with the constant *IPC_RMID* to delete the message queue.

(Please note: Just like the *ipcs* command displays IPC objects, the *ipcrm* command can be used to delete System V IPC objects from the shell prompt.)

The other functions are System V message-queue-related functions (browse the manual pages to learn more about them).

Shared memory and semaphores

Processes normally do not share memory. The memory of a process image (like stack, global data and heap) belongs to that process, and cannot be shared, as we saw when discussing *fork* in the processes article. But you can create memory that can be shared between multiple processes.